

---

**CONCEPTO  
ARQUITECTURA EMPRESARIAL  
IFRAME SEGURO DE CAPTURA Y CAMBIO DE PIN  
— PLATAFORMA PREPAGO  
CREDIBANCO  
VERSIÓN 1.0**

---

## HISTORIA DE REVISIONES

---

| Fecha      | Versión | Descripción     | Autor                     |
|------------|---------|-----------------|---------------------------|
| 29/04/2026 | 1.0     | Versión Inicial | Javier Ricardo<br>Agudelo |
|            |         |                 |                           |
|            |         |                 |                           |

# TABLA DE CONTENIDO

---

|                                             |    |
|---------------------------------------------|----|
| HISTORIA DE REVISIONES .....                | 2  |
| TABLA DE CONTENIDO .....                    | 3  |
| GUÍA DE USO .....                           | 4  |
| NECESIDAD.....                              | 5  |
| PREMISAS DEL CONCEPTO .....                 | 6  |
| LINEAMIENTOS .....                          | 7  |
| RESTRICCIONES .....                         | 8  |
| ARQUITECTURA PROPUESTA .....                | 9  |
| Diagrama de Arquitectura.....               | 9  |
| Descripción de componentes .....            | 10 |
| Descripción del proceso.....                | 11 |
| Aplicaciones y/o soluciones afectadas ..... | 12 |
| ANEXOS.....                                 | 13 |

## GUÍA DE USO

---

El presente documento detalla el concepto del equipo de arquitectura empresarial generado para la necesidad expuesta por el negocio. Se identifica la necesidad, premisas, lineamientos, restricciones y se detalla la arquitectura propuesta.

A su vez, contiene el detalle de los elementos evaluados y el análisis a alto nivel de los posibles componentes involucrados dentro de Credibanco.

### **Cómo diligenciar este documento:**

- *No se deben eliminar las anotaciones al inicio de cada sección con información de lo que debe contener cada punto. Estas notas sirven para ayudar a determinar la información a rellenar y para ayudar a las personas que revisan el documento a saber que debe estar cumplimentado en cada apartado.*
- *No se deben eliminar puntos del índice o capítulos del documento. Si algún punto no aplica se pondrá el comentario "NO APLICA".*
- *No olvide actualizar la fecha y la versión del documento en el encabezado, y el nombre del archivo en el pie de página.*
- *Tenga la precaución de revisar todos los encabezados y pies de página del documento para confirmar que la información que figura en los mismos está correctamente actualizada, ya que hay encabezados y pies de página para diferentes secciones del documento, y para páginas pares e impares.*

## NECESIDAD

---

El presente concepto se enfoca en la definición de la arquitectura para la captura segura de PIN (asignación y cambio) de tarjetas prepago dentro de la plataforma Prepago On-Prem de Credibanco, cumpliendo con PCI DSS v4.0.

1. Se requiere un mecanismo seguro para que el tarjetahabiente (TH) pueda asignar un PIN al activar su tarjeta prepago (física o virtual) y realizar cambios de PIN posteriores, sin que el PIN en claro sea expuesto a ningún componente fuera del HSM.
2. Se requiere que el flujo de captura de PIN funcione tanto para tarjetas nominadas (autenticadas vía portal TH con SSO + MFA) como para tarjetas innominadas (autenticadas por Proof of Possession: últimos 4 dígitos + fecha vencimiento + CVV2).
3. Se requiere que el componente de captura de PIN sea un iframe embebible en el Portal Tarjetahabiente, en un dominio separado controlado por Credibanco, de forma que el portal de la entidad emisora quede fuera del scope CDE de PCI DSS.
4. Se requiere que el PIN sea cifrado end-to-end desde el navegador del TH hasta el HSM Thales payShield, utilizando criptografía asimétrica RSA-2048 con llaves efímeras, y que el HSM realice el interchange (re-cifrado) con la llave pública del proveedor (Pomelo) antes de enviar el PIN al procesador.

## PREMISAS DEL CONCEPTO

---

Esta sección presenta las premisas que rigen la arquitectura propuesta.

1. Credibanco cuenta con un HSM Thales payShield (9000 o 10K) en configuración HA Cluster, dentro del perímetro CDE, con los comandos EI/EJ, GI/GJ, EO/EP y FK/FL habilitados en el firmware.
2. El HSM Thales es stateless por diseño: no retiene llaves entre llamadas. La llave privada efímera generada por el comando EI sale cifrada bajo la LMK (Local Master Key) del HSM y se almacena temporalmente en Redis (TTL 5 minutos) por el backend.
3. La llave pública de Pomelo está precargada en el HSM mediante el comando FK, bajo un índice fijo, con rotación periódica programada según acuerdo con el proveedor.
4. El Portal Tarjetahabiente existe como una SPA Angular 17+ desplegada en OpenShift, con autenticación SSO + MFA para tarjetas nominadas.
5. Para tarjetas innominadas, se dispone de un micro-portal de activación público (URL genérica comunicada en el carrier) que autentica por Proof of Possession.
6. La comunicación entre el backend y el HSM se realiza por red interna con mTLS (PCI DSS v4.0 Req 4.2.1).
7. La comunicación entre el backend y Pomelo se realiza por VPN con mTLS y autenticación por headers propietarios.
8. El navegador del TH soporta Web Crypto API (SubtleCrypto) — compatible con Chrome, Firefox, Edge, Safari en versiones actuales.

## LINEAMIENTOS

---

Esta sección presenta los lineamientos que rigen la arquitectura propuesta.

1. El PIN en claro NUNCA debe existir fuera del HSM. En el navegador existe solo durante la captura (milisegundos) y se borra inmediatamente con `Uint8Array.fill(0)`.
2. El backend actúa como proxy opaco: transporta el ciphertext sin capacidad de descifrarlo. Queda fuera del scope CDE para el manejo de PIN.
3. Todo el flujo debe ser trazable end-to-end mediante un `correlationId` que viaja por las 4 capas (iframe → backend → HSM → Pomelo).
4. Se debe implementar idempotencia en la llamada al proveedor mediante `idempotencyKey` para evitar doble asignación/cambio de PIN.
5. Se debe implementar protección anti-replay mediante nonce incluido en el bloque PIN antes del cifrado.
6. El iframe debe estar en un dominio separado controlado por Credibanco (ej: `secure-pin.credibanco.com`) con CSP headers estrictos y SRI en todos los scripts.
7. Se debe implementar rate limiting: máximo 3 sesiones de PIN por tarjeta por hora, para proteger el HSM de agotamiento de recursos.
8. El teclado de captura de PIN debe tener: `autocomplete="off"`, `oncopy/onpaste` bloqueados, timeout de 30 segundos por inactividad, y máximo 3 intentos por sesión.

## RESTRICCIONES

---

Esta sección presenta las restricciones que rigen la arquitectura propuesta.

1. No se puede utilizar pinpad físico ni ATM para la asignación de PIN en el canal digital.
2. No se puede enviar el PIN por SMS, email ni ningún canal no cifrado (restricción PCI DSS).
3. El HSM Thales no soporta el comando GK (translate RSA-to-RSA atómico) en el firmware actual — se requieren dos llamadas separadas: GI (descifrar) + EO (re-cifrar).
4. El iframe no puede prevenir screenshots del navegador (limitación del sistema operativo). Se mitiga con TTL corto de la sesión (5 minutos) y auto-limpieza del DOM.
5. La latencia máxima aceptable para el flujo completo (captura → cifrado → interchange → Pomelo → respuesta) es de 5 segundos. El circuit breaker del backend tiene timeout de 5 segundos hacia el HSM.



# ARQUITECTURA PROPUESTA

Esta sección describe la arquitectura propuesta de la solución a las necesidades planteadas en la sección Necesidades.

## Diagrama de Arquitectura

El diagrama está organizado en 4 swimlanes que representan las capas del flujo:

- Angular iframe (captura de PIN)
- Backend portal Credibanco (proxy seguro, fuera CDE)
- HSM Credibanco Thales payShield (CDE, interchange de llaves)
- Proveedor de cambio de PIN / Pomelo (API JSON, TLS 1.3 / mTLS)

El flujo se divide en 5 fases secuenciales (Fase 0 a Fase 4).

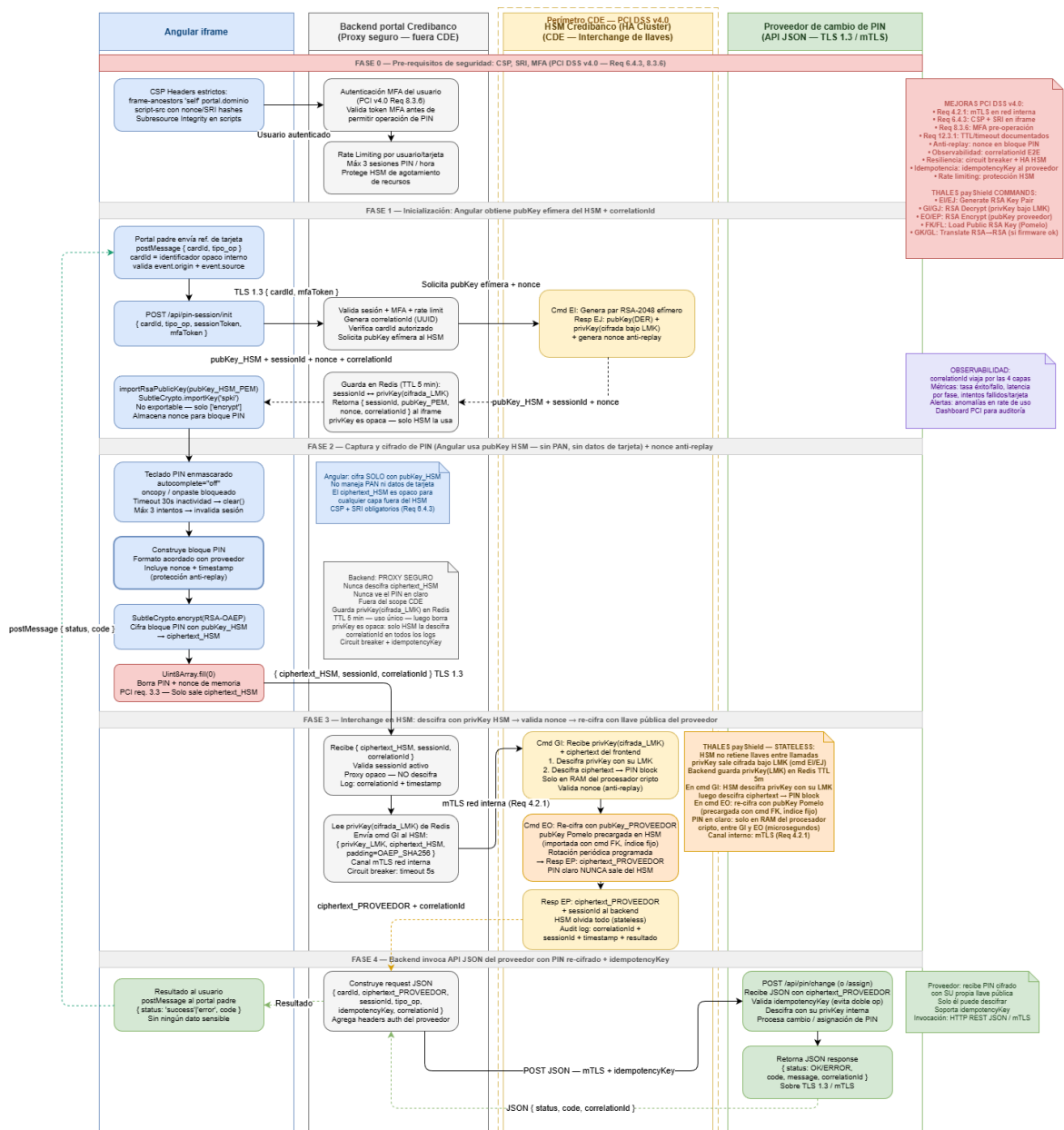


Diagrama 1. – Diagrama de Arquitectura

## Descripción de componentes

| Identificador | Componente                               | Descripción                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CSA-001       | Angular iframe (Captura PIN)             | SPA Angular 17+ desplegada en dominio separado (`secure-pin.credibanco.com`). Recibe la pubKey efímera del HSM, captura el PIN del TH en un teclado enmascarado, cifra el PIN con SubtleCrypto RSA-OAEP, y envía el ciphertext al backend. Nunca almacena el PIN — lo borra de memoria inmediatamente después de cifrar. CSP + SRI obligatorios (Req 6.4.3).                                                       |
| CSA-002       | Backend Portal Credibanco (Proxy Seguro) | Spring Boot 3 + Java 21 desplegado en OpenShift. Actúa como proxy opaco entre el iframe y el HSM. Genera correlationId, valida sesión + MFA, aplica rate limiting, almacena la privKey(cifrada_LMK) en Redis con TTL 5 min. NUNCA descifra el ciphertext del PIN. Fuera del scope CDE para PIN. Circuit breaker con Resilience4j (timeout 5s, 1 retry).                                                            |
| CSA-003       | HSM Thales payShield (HA Cluster)        | Hardware Security Module dentro del perímetro CDE. Genera pares RSA efímeros (cmd EI/EJ), descifra el ciphertext del frontend (cmd GI/GJ usando la privKey bajo LMK), y re-cifra con la pubKey de Pomelo precargada (cmd EO/EP). El PIN en claro existe SOLO en la RAM protegida del procesador criptográfico del HSM, entre los comandos GI y EO (microsegundos). Es stateless: no retiene llaves entre llamadas. |
| CSA-004       | Redis 7+ (Sesión efímera)                | Cache en memoria para almacenar: sessionId ↔ privKey(cifrada_LMK) con TTL 5 minutos y uso único. También almacena contadores de rate limiting por tarjeta. La privKey almacenada es opaca — solo el HSM puede descifrarla con su LMK.                                                                                                                                                                              |
| CSA-005       | Pomelo API (Proveedor)                   | API REST JSON del proveedor de procesamiento. Recibe el PIN cifrado con su propia llave pública (ciphertext_PROVEEDOR), lo descifra internamente, y asigna/cambia el PIN de la tarjeta. Soporta idempotencyKey. Comunicación por VPN + mTLS + headers de autenticación propietarios.                                                                                                                               |
| CSA-006       | SSO (Identity Provider)                  | Sistema de autenticación que valida la sesión del TH (para tarjetas nominadas) o la sesión efímera del micro-portal (para innominadas). Emite JWT tokens con MFA claim. OAuth2 / OIDC.                                                                                                                                                                                                                             |

Tabla 1. – Listado Responsabilidades de Componentes

## Descripción del proceso

### Fase 0 — Pre-requisitos de seguridad:

El portal valida que el usuario tenga MFA activo (Req 8.3.6). Se verifican CSP headers y SRI en el iframe. Se aplica rate limiting (máx 3 sesiones PIN/hora por tarjeta).

### Fase 1 — Inicialización:

El portal padre envía ``postMessage({ cardId, tipo_op })`` al iframe. El iframe solicita al backend una sesión de PIN. El backend valida la sesión, genera un `correlationId`, y solicita al HSM una `pubKey` efímera (cmd EI). El HSM genera un par RSA-2048, devuelve la `pubKey` en DER y la `privKey` cifrada bajo LMK. El backend almacena la `privKey(LMK)` en Redis (TTL 5 min) y retorna la `pubKey + nonce + sessionId` al iframe.

### Fase 2 — Captura y cifrado de PIN:

El iframe importa la `pubKey` con ``SubtleCrypto.importKey('spki')`` marcada como no exportable. El TH digita su PIN en el teclado enmascarado (`autocomplete=off`, `timeout 30s`). El iframe construye un bloque PIN (`PIN + nonce + timestamp`), lo cifra con ``SubtleCrypto.encrypt(RSA-OAEP)`` → `ciphertext_HSM`. Inmediatamente borra el PIN de memoria con ``Uint8Array.fill(0)``. Solo el `ciphertext_HSM` sale del iframe.

### Fase 3 — Interchange en HSM:

El backend recibe el `ciphertext_HSM` como proxy opaco (no lo descifra). Lee la `privKey(cifrada_LMK)` de Redis y la envía al HSM junto con el `ciphertext`. El HSM ejecuta cmd GI: descifra la `privKey` con su LMK, luego descifra el `ciphertext` → obtiene el bloque PIN en claro en su RAM protegida. Valida el nonce (anti-replay). Ejecuta cmd EO: re-cifra el PIN con la `pubKey` de Pomelo (precargada con cmd FK) → `ciphertext_PROVEEDOR`. El PIN en claro existió solo en la RAM del HSM entre GI y EO. El HSM devuelve el `ciphertext_PROVEEDOR` al backend.

### Fase 4 — Envío al proveedor:

El backend construye el request JSON con el `ciphertext_PROVEEDOR + idempotencyKey + correlationId` y lo envía a Pomelo por mTLS. Pomelo descifra con su `privKey` interna, asigna/cambia el PIN, y retorna confirmación. El backend notifica al iframe el resultado. El iframe envía ``postMessage({ status, code })`` al portal padre (sin datos sensibles).

## Aplicaciones y/o soluciones afectadas

| Aplicación                               | Impacto                                                                                                  |
|------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Portal Tarjetahabiente (Angular)         | Debe integrar el iframe de PIN como componente embebido. Comunicación por postMessage.                   |
| Micro-portal de Activación (innominadas) | Debe integrar el mismo iframe de PIN después del Proof of Possession.                                    |
| Backend Prepaid Cardholder API           | Debe exponer endpoints `/api/pin-session/init` y `/api/pin-session/submit`. Integración con HSM y Redis. |
| HSM Thales payShield                     | Requiere habilitación de comandos EI/EJ, GI/GJ, EO/EP. Precarga de pubKey Pomelo con cmd FK.             |
| Redis                                    | Nuevo uso: almacenamiento de privKey(cifrada_LMK) con TTL 5 min + contadores rate limiting.              |
| Pomelo API                               | Integración existente. Nuevo endpoint: POST /api/pin/assign y POST /api/pin/change.                      |
| OpenShift                                | Nuevo deployment para el iframe Angular en dominio separado (secure-pin.credibanco.com).                 |

## ANEXOS

---

<Se incluye documentos de referencia que hacen parte del diseño de Arquitectura >

<Autores: Arquitecto>

| NOMBRE | OBSERVACIONES                  |
|--------|--------------------------------|
|        | <Link de referencia del anexo> |
|        |                                |

Tabla 2.